[승코의 차근차근 Flutter] 02. 다트 객체지향 프로그래밍

📦 1. 클래스(Class): 설계도

의미:

클래스는 객체를 만들기 위한 틀, 설계도입니다. 이 안에 변수(속성)와 함수(기능)를 넣을 수 있어요.

```
class Idol {
  String name = '블랙핑크';

  void sayName() {
    print('저는 $name입니다.');
  }
}
```

쉽게 말해:

- `Idol`은 '아이돌 그룹'이라는 설계도
- `name`은 이 아이돌의 이름
- `sayName()`은 아이돌이 이름을 말하는 기능

🧪 2. 인스턴스(Instance): 클래스로 만든 실제 객체

```
void main() {
  Idol blackPink = Idol(); // Idol 클래스 인스턴스 생성
  blackPink.sayName();     // 기능 사용
}
```

- `Idol()`을 호출하면 실제 `blackPink`라는 객체가 생깁니다.
- 이렇게 클래스를 사용해서 만든 '진짜 객체'를 인스턴스라고 불러요.

🔧 3. 생성자(Constructor): 객체를 만들 때 자동 실행되는 함수

```
class Idol {
  final String name;

  Idol(this.name); // 생성자
}
```

- 객체 생성 시 초기값을 넣고 싶을 때 사용합니다.
- `this.name = name`은 왼쪽은 클래스 내부의 변수, 오른쪽은 생성자 매개변수

```
void main() {
  Idol bts = Idol('방탄소년단');
}
```

📛 4. 네임드 생성자(Named Constructor): 이름 붙인 생성자

```
class Idol {
  final String name;
  final int membersCount;

  Idol(this.name, this.membersCount);

  Idol.fromMap(Map<String, dynamic> map)
      : name = map['name'],
        membersCount = map['membersCount'];
}
```

- 다양한 방식으로 객체를 만들고 싶을 때 쓰는 “보조 생성자”입니다.

```
void main() {
  var bts = Idol.fromMap({'name': 'BTS', 'membersCount': 7});
}
```

🔒 5. 프라이빗 변수(Private Variable)

```
class Idol {
  String _name;

  Idol(this._name);
}
```

- `_name` 처럼 변수 앞에 **밑줄(_)** 을 붙이면 프라이빗입니다.
- 프라이빗 변수는 **같은 파일 안에서만 접근 가능**, 다른 파일에서는 접근 불가!

🚪 6. 게터 / 세터 (Getter / Setter)

```
class Idol {
  String _name = '블랙핑크';
```

```
  String get name => _name; // 읽기
  set name(String name) => _name = name; // 쓰기
}
```

```
void main() {
  Idol idol = Idol();
  idol.name = '에이핑크';  // 세터 사용
  print(idol.name);       // 게터 사용
}
```

요점:

- 게터는 `값을 꺼낼 때`
- 세터는 `값을 넣을 때`
- 직접 변수에 접근하지 못하도록 **우회통로** 역할

👫 7. 상속(Inheritance): 부모 클래스의 기능을 물려받기

```
class Idol {
  final String name;
  final int membersCount;

  Idol(this.name, this.membersCount);

  void sayName() => print('저는 $name입니다.');
}

class BoyGroup extends Idol {
  BoyGroup(String name, int membersCount) : super(name, membersCount);

  void sayMale() => print('저는 남자 아이돌입니다.');
}
```

- `BoyGroup`은 `Idol`을 **상속**받아 기능을 그대로 사용할 수 있습니다.
- `super`는 부모 클래스 의미

✏️ 8. 오버라이드(Override): 부모 기능을 바꾸기

```
class GirlGroup extends Idol {
  GirlGroup(super.name, super.membersCount);

  @override
  void sayName() {
    print('저는 여자 아이돌 $name입니다.');
  }
}
```

- 부모 클래스의 메서드를 **자식이 재정의**
- `@override`는 꼭 붙이지 않아도 되지만, 붙이는 게 좋음 (명확성)

🤝 9. 인터페이스(Interface)

```
class GirlGroup implements Idol {
  final String name;
  final int membersCount;

  GirlGroup(this.name, this.membersCount);

  void sayName() => print('저는 여자 아이돌 $name입니다.');
  void sayMembersCount() => print('$name 멤버는 $membersCount명입니다.');
}
```

- `implements`를 쓰면 **Idol의 메서드를 반드시 구현해야 함**
- 상속은 기능을 “물려받는 것”, 인터페이스는 “형식을 지키는 것”
- 하나의 클래스가 여러 인터페이스를 implement할 수 있음

🧩 10. 믹스인(Mixin): 기능 조각만 섞기

```
mixin IdolSingMixin on Idol {
  void sing() => print('$name이 노래합니다.');
}

class BoyGroup extends Idol with IdolSingMixin {
  BoyGroup(super.name, super.membersCount);
}
```

- `with` 키워드를 사용해서 특정 기능만 골라서 붙일 수 있음
- 상속처럼 **모든 기능**을 받을 필요 없이, 필요한 기능만 사용 가능
- `on Idol` 은 Idol을 상속한 클래스에서만 이 믹스인 사용 가능

🔍 11. 추상 클래스(Abstract Class)

```
abstract class Idol {
  final String name;
  final int membersCount;

  Idol(this.name, this.membersCount);

  void sayName(); // 추상 메서드
  void sayMembersCount(); // 추상 메서드
}
```

```
class GirlGroup implements Idol {
  final String name;
  final int membersCount;

  GirlGroup(this.name, this.membersCount);

  void sayName() => print('저는 여자 아이돌 $name입니다.');
```

```
  void sayMembersCount() => print('$name 멤버는 $membersCount명입니다.');
}
```

- `abstract`는 직접 인스턴스화 불가능
- 자식 클래스는 추상 메서드를 **반드시** 구현해야 함

🧬 12. 제네릭(Generic): 타입을 나중에 정하는 방식

```
class Cache<T> {
  final T data;
  Cache({required this.data});
}

void main() {
  var cache = Cache<List<int>>(data: [1, 2, 3]);
  print(cache.data.reduce((a, b) => a + b));
}
```

- `<T>`는 타입 파라미터 (T, E, K, V 등 자유롭게)
- 클래스를 사용할 때 타입을 지정해서 다양하게 활용 가능
- 예: `List<String>`, `Map<String, int>`

📌 13. 스태틱(Static): 클래스 전체에서 공유되는 변수/함수

```
class Counter {
  static int i = 0;

  Counter() {
    i++;
    print(i);
  }
}
```

```
void main() {
  Counter(); // 1
  Counter(); // 2
  Counter(); // 3
}
```

- `static` 은 클래스 자체에 귀속되는 변수/함수
- 인스턴스를 여러 개 만들어도 **같은 값 공유**

✅ 요약 키워드 정리

개념	핵심 키워드
클래스	설계도, 변수(속성), 함수(기능)
생성자	초기화, this, 클래스명과 동일
네임드 생성자	fromMap, 여러 생성자 지원
프라이빗	_name, 파일 단위 보호
게터/세터	get/set, 값 읽고 쓰는 통로
상속	extends, super, 기능 물려받기
오버라이드	@override, 부모 기능 재정의
인터페이스	implements, 형식 보장, 반드시 구현
믹스인	with, 기능 덧붙이기
추상	abstract, 인스턴스화 불가, 강제 구현
제네릭	, 타입 유연성
스태틱	static, 공유 변수/함수